Name : Kirthetharan A/L Sehkar
Matric No : AI240016

Lab #4a:
CLIENT-SIDE SCRIPTING (JAVASCRIPT)

Item	Description
Topic	HTML and modern client-side JavaScript: functions, events, conditions, loops and DOM manipulation
Domain of Learning	Psychomotor (P2: Set; P3: Guided Response; P4: Mechanism); Affective (A1 and A2, subject to approved course CLO mapping)
Learning Objective	Develop interactive web pages using JavaScript functions, event handling, input validation, loops and DOM methods, following readable and safe coding practices.
Lab Activity Objective	Apply JavaScript to update page content, validate user input and dynamically generate HTML elements; demonstrate and explain the implemented solutions.

General Instructions

- Answer all three questions.
- Write working HTML5 and JavaScript that runs in a current desktop browser without external libraries.
- Use an external JavaScript file (script.js) where practical, or a script element with defer-aware placement. Prefer addEventListener() to inline onclick attributes.
- Use textContent for user-facing text. Do not use innerHTML to insert user input.
- Make your output readable and accessible: descriptive labels, meaningful button text and a visible status/output region.
- Develop and test each question independently. Retain evidence of your code and browser output.

Question 1 — Interactive Text Changer (5 marks)

Create a webpage that changes a paragraph when the user clicks a button. Start from the HTML below and write the JavaScript function and event listener.

```
<div id="myElement">
```

```
  <p id="message">Welcome to my website!</p>
</div>
<button id="changeBtn" type="button">Change Message</button>
```

- Select the paragraph and button using JavaScript DOM selection methods. (1 mark)
- Define a function named **changeMessage()** that replaces the paragraph text with **"Hello there!"**. (2 marks)
- Attach the function to the button using **addEventListener("click", ...)**. (2 marks)

Expected result:
On page load, the original message is displayed. When **"Change Message"** is clicked, the paragraph displays **"Hello there!"**.

Index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Q1 - Interactive Text Changer</title>
  <!-- defer: script runs after the HTML is parsed, so the elements already exist -->
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Question 1: Interactive Text Changer</h1>

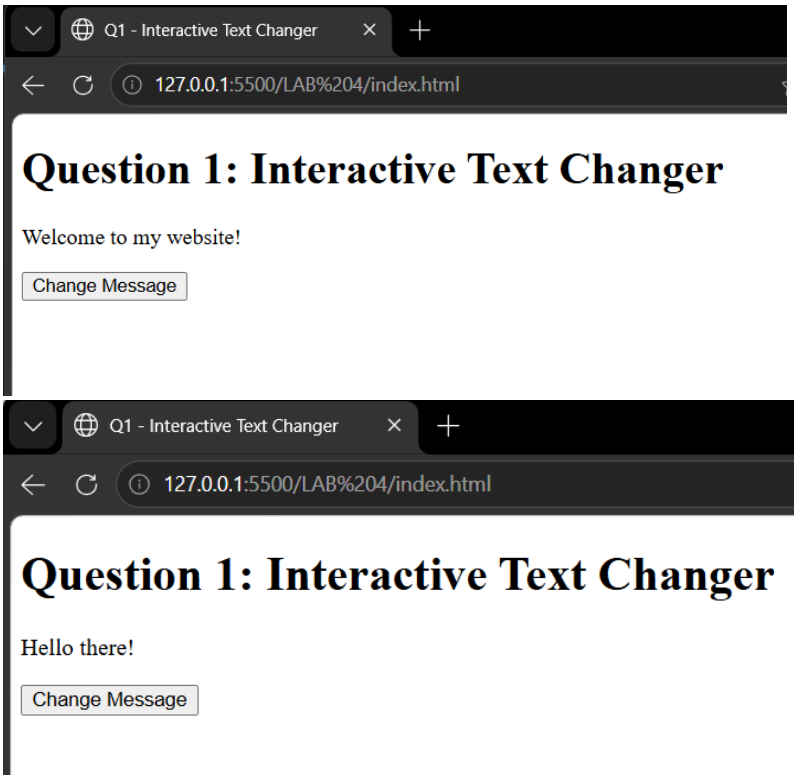
  <div id="myElement">
    <p id="message">Welcome to my website!</p>
  </div>
  <button id="changeBtn" type="button">Change Message</button>
</body>
</html>
```

Script.js

```javascript
// 1. Select the paragraph and the button using DOM selection methods
const message = document.getElementById("message");
const changeBtn = document.getElementById("changeBtn");

// 2. Function that replaces the paragraph text
function changeMessage() {
  message.textContent = "Hello there!";
}

// 3. Attach the function to the button's click event
changeBtn.addEventListener("click", changeMessage);
```



Question 2 - Smart Number Validator (5 marks)

Create an interactive webpage that accepts a number and displays a validation message below the input. Use a function named **checkInput()** and an if...else if...else structure.

```
<label for="userInput">Enter a number:</label>
<input type="text" id="userInput" placeholder="Enter a number">
<button id="checkBtn" type="button">Check</button>
<p id="result" role="status" aria-live="polite"></p>
```

- Read the input and handle empty or whitespace-only values as invalid. (1 mark)
- Convert a valid numeric string to a number and detect invalid values (e.g., abc). (1 mark)
- Display the correct message for numbers below, above or equal to 10. (2 marks)
- Display the result in #result using textContent and register the click event with addEventListener(). (1 mark)

Expected messages:

"The number is lower than 10", "The number is higher than 10", "The number is equal to 10", or "Please enter a valid number!".

Test with: 5, 10, 15, abc, and an empty input. Decimal numbers such as 10.5 are valid.

Index.html

```
LAB 4 > Q2 > index.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Q2 - Smart Number Validator</title>
  <!-- defer: script runs after the HTML is parsed, so the elements already exist -->
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Question 2: Smart Number Validator</h1>

  <label for="userInput">Enter a number:</label>
  <input type="text" id="userInput" placeholder="Enter a number">
  <button id="checkBtn" type="button">Check</button>
  <p id="result" role="status" aria-live="polite"></p>
</body>
</html>

```

Script.js

```
// Select the input, button and output paragraph
const userInput = document.getElementById("userInput");
const checkBtn = document.getElementById("checkBtn");
const result = document.getElementById("result");

function checkInput() {
  // Read the input and remove spaces at the start and end
  const value = userInput.value.trim();

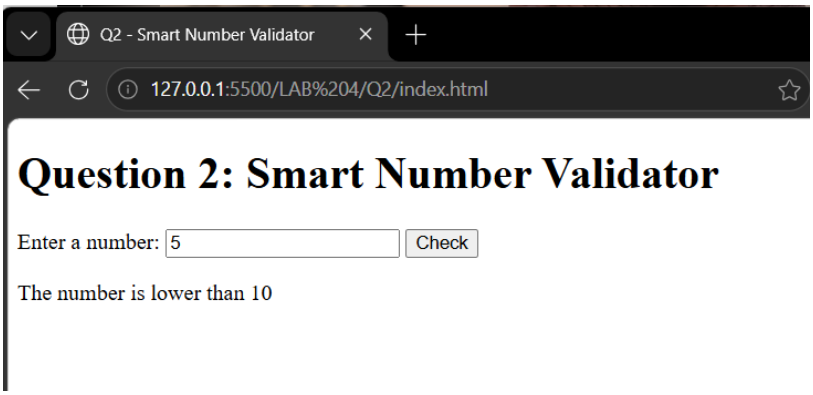
  // Convert the text to a number (e.g. "10.5" -> 10.5, "abc" -> NaN)
  const number = Number(value);

  if (value === "" || !Number.isFinite(number)) {
    // Empty, whitespace-only, or not a valid number (e.g. "abc")
    result.textContent = "Please enter a valid number!";
  } else if (number < 10) {
    result.textContent = "The number is lower than 10";
  } else if (number > 10) {
    result.textContent = "The number is higher than 10";
  } else {
    result.textContent = "The number is equal to 10";
  }
}

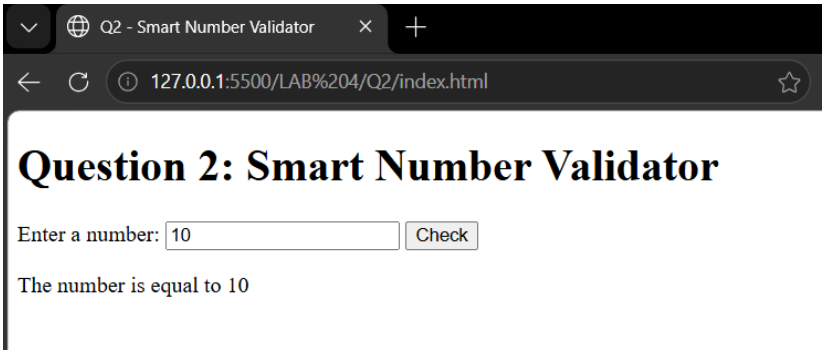
// Register the click event
checkBtn.addEventListener("click", checkInput);

```

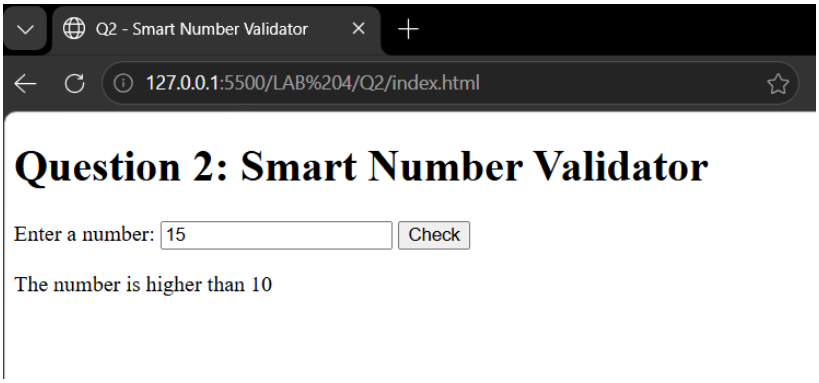
Test with number 5:



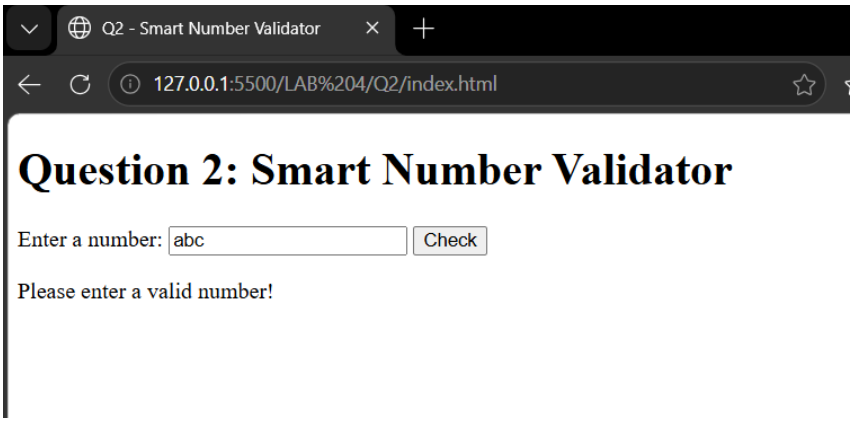
Test with number 10:



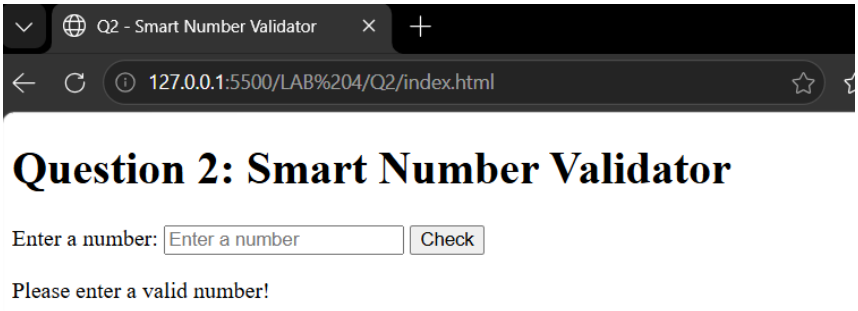
Test with number 15:



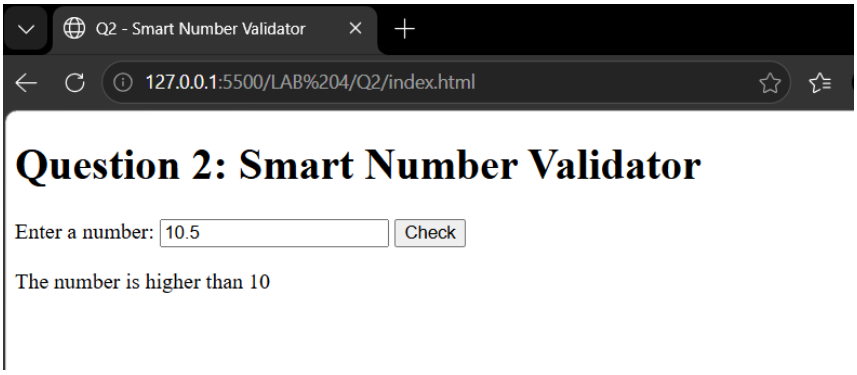
Test with abc :



Test with empty input :



Test with decimal number 10.5



Question 3 - Dynamic Number Generator (5 marks)

Write a JavaScript function named **generateNumbers()** to display a numbered list from 1 to 5 when the user clicks **"Generate"**. Each number must be created as a new list item.

```
<button id="generateBtn" type="button">Generate</button>
<ol id="numberList"></ol>
```

- Create the function and attach it to the button using **addEventListener()**. (1 mark)
- Use a for loop that iterates from 1 through 5. (2 marks)
- Create each <li> with **document.createElement()**, set its text with textContent, and append it to #numberList. (1 mark)
- Clear existing items before generating again so repeated clicks do not duplicate the list. (1 mark)

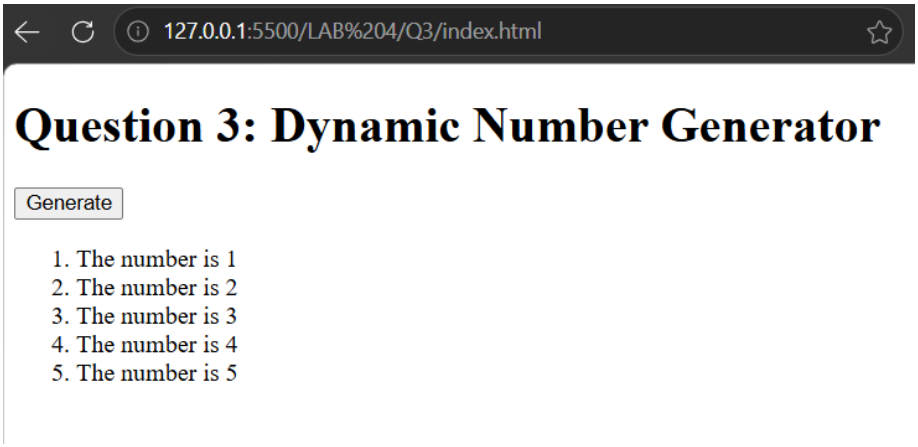
Expected result:
An ordered list containing "The number is 1" through "The number is 5", with no duplicate items after repeated clicks.

Index.html



Script.js

```
3
4   // Select the button and the ordered list
5   const generateBtn = document.getElementById("generateBtn");
6   const numberList = document.getElementById("numberList");
7
8   function generateNumbers() {
9     // Clear any existing items so repeated clicks do not duplicate the list
10    numberList.replaceChildren();
11
12    // Loop from 1 through 5
13    for (let i = 1; i <= 5; i++) {
14      const item = document.createElement("li");   // create a new <li>
15      item.textContent = "The number is " + i;     // set its text safely
16      numberList.appendChild(item);                // add it to the list
17    }
18  }
19
20  // Attach the function to the button's click event
21  generateBtn.addEventListener("click", generateNumbers);
22
```



Submission and Demonstration

- Create a GitHub repository titled bic21203-lab4a-javascript (or use the repository specified by your lecturer). Include separate files or folders for Q1, Q2 and Q3.
- Add a README.md containing your name/student ID, a brief explanation, instructions to run each question and screenshots of the outputs.
- Submit a single PDF containing a cover page, GitHub repository link, concise test evidence for each question and a short reflection (3–5 sentences). Do not paste all source code into the PDF; the repository serves as the source-code record.
- Submit the PDF through AUTHOR (or the official LMS specified by the lecturer). Be prepared for a short live demonstration and questions on your code.

Assessment:

CLO2 — Programming implementation (15 marks); CLO3 — Demonstration and understanding (5 marks).

Assessment Rubrics — Lab 4a

Assessment uses criterion-referenced descriptors. Award an integer score from 0 up to the criterion maximum based on observable evidence; partial credit is permitted. CLO mappings are retained from the original lab and should be confirmed against the approved course outline.

A. JavaScript Functionality and Code Quality (CLO2 — Psychomotor): 15 marks

Criteria	**Excellent (5)**	**Good (4)**	**Satisfactory (3)**	**Needs Improvement (1-2)**	**Unsatisfactory (0)**	**Marks**
Use of Functions (P4)	Creates and correctly uses functions and event listeners for Q1 and Q3; button actions update the page.	Functions and event listeners work with minor issues.	Basic functions work, but one event or required action is incomplete.	Functions/events are partially implemented and unreliable.	No functional JavaScript functions or event handling.	___/5 × 5
Use of For Loops (P4)	Correct for loop generates five list items with DOM methods and prevents duplicate output.	Loop generates the list; minor DOM or repeat-click issue.	Loop works, but list output or DOM manipulation is incomplete.	Partial loop; incorrect range or mostly nonfunctional output.	No functioning for loop or generated output.	___/5 × 6
Code Quality (P2)	Well-organized, readable code; correct input validation for Q2, appropriate DOM text updates and meaningful names.	Readable code and mostly correct Q2 validation, with minor defects.	Functional code but inconsistent structure or missing input cases.	Poor organization and substantial Q2 validation errors.	Unusable code; Q2 validation not implemented.	___/5 × 4

Total	/15

CLO2 weights retained: Functions (5), For Loops (6), Code Quality (4). Input validation is assessed within Code Quality; event handling and DOM manipulation are assessed within Functions and For Loops.

B. Presentation and Understanding of Concepts (CLO3 — Affective): 5 marks

Criteria	**Excellent (5)**	**Good (4)**	**Satisfactory (3)**	**Needs Improvement (1-2)**	**Unsatisfactory (0)**	**Marks**
Clarity and Confidence (A1)	Clearly demonstrates all three tasks and test cases; speaks confidently and communicates results effectively.	Clear demonstration with minor omissions or confidence issues.	Adequate demonstration but some outcomes are not clearly shown.	Limited demonstration, unclear explanations or low engagement.	Unable to demonstrate working outputs or communicate results.	___/5 × 2
Explanation of JavaScript Concepts (A2)	Accurately explains functions, events, conditions, validation, loops and DOM methods; justifies choices.	Good explanation of most concepts with minor gaps.	Basic understanding with limited justification of code choices.	Limited understanding; struggles to explain key JavaScript concepts.	Unable to explain the JavaScript concepts used.	___/5 × 3
					Total	/5

OVERALL TOTAL: CLO2 (15) + CLO3 (5) = 20 MARKS